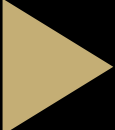




CS-218 Data Structures and Algorithms



LECTURE 22

BY UROOJ AINUDDIN

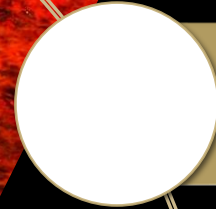




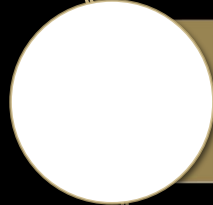
Trees

BOOK 1 CHAPTER 13

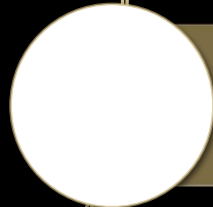




We implemented code to determine if a binary tree is an ACBT.



We were introduced to depth-first tree traversal algorithms.



We implemented code for inorder traversal.



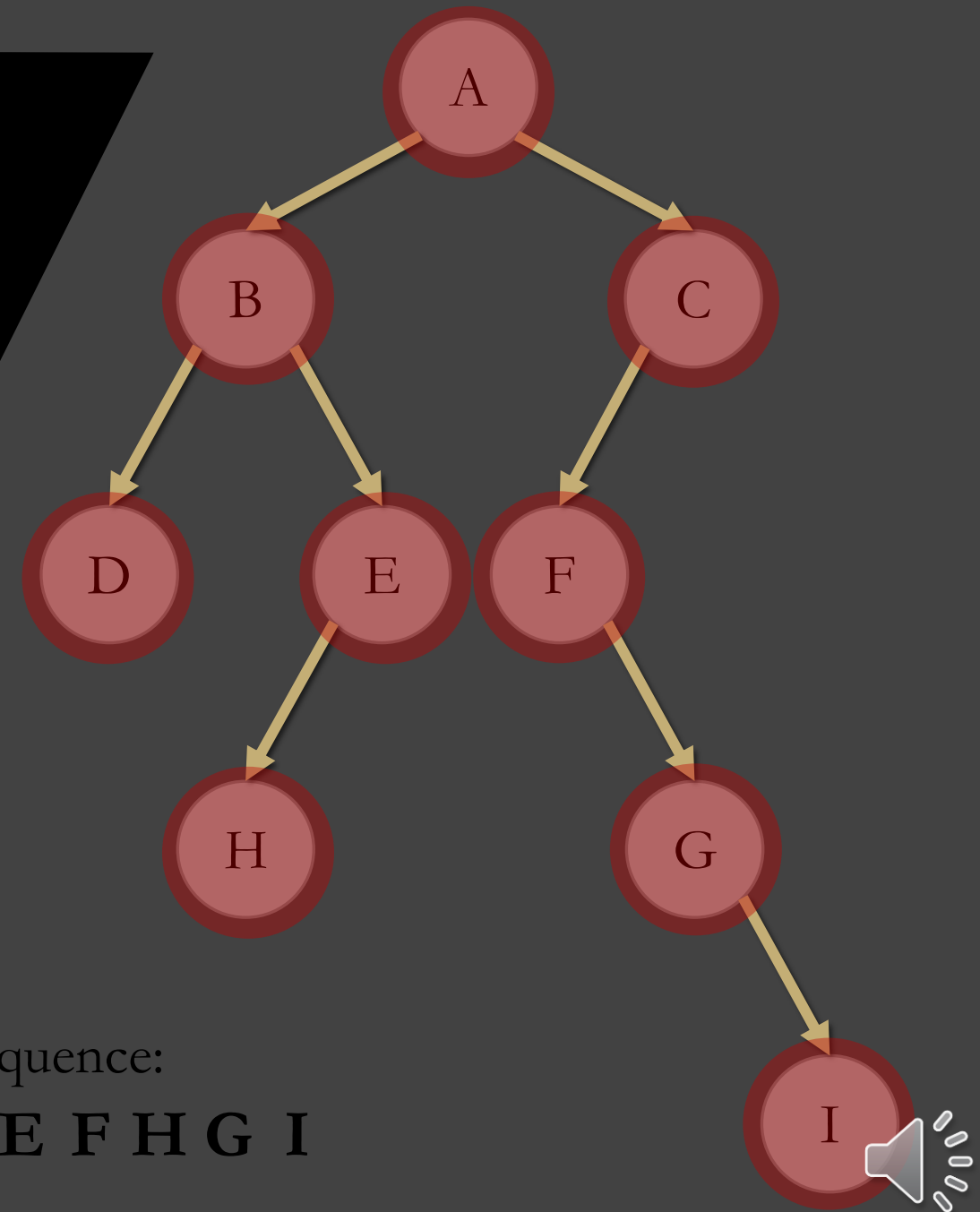
We examined all codes for time and space complexities.

In the last lecture...



Breadth-first traversal

- Start at the root.
- Within the same level, visit nodes from left to right.
- When all nodes in level k have been visited, move to level $k+1$.
- We print the data field of a node when we **visit** it.
- This traversal is like reading English text on a page.



The binarytree.py file - traversebf

```
def traversebf(self):
    nodes = [self]
    print(self.data, end=" ")
    i = 0 #index of element in nodes
    n = 1 #number of elements in nodes
    while i < n:
        p = nodes[i]
        if p.lc is not None:
            print(p.lc.data, end=" ")
            nodes.append(p.lc)
            n += 1
        if p.rc is not None:
            print(p.rc.data, end=" ")
            nodes.append(p.rc)
            n += 1
        i += 1
    print()
```

- The while loop iterates as many times as the number of nodes in the tree.
- The body of the while takes $O(1)$ time.

Time: $O(n)$



The binarytree.py file - traversebf

```
def traversebf(self):
    nodes = [self]
    print(self.data, end=" ")
    i = 0 #index of element in nodes
    n = 1 #number of elements in nodes
    while i < n:
        p = nodes[i]
        if p.lc is not None:
            print(p.lc.data, end=" ")
            nodes.append(p.lc)
            n += 1
        if p.rc is not None:
            print(p.rc.data, end=" ")
            nodes.append(p.rc)
            n += 1
        i += 1
    print()
```

- Two integer variables **i** and **n**, a Btree object **p** and a Python list **nodes** is used.
- The list **nodes** contains as many elements as the number of nodes when the loop completes. So, its maximum size is **n**.

Space: $O(n)$



```
def buildtree(val):
    if len(val) == 0 or val[0] is None:
        return None
    root = BTree(val[0])
    nodes = [root]
    n = 0 #keeps track of item in nodes
    i = 1 #keeps track of item in val
    while i < len(val):
        p = nodes[n]
        if val[i] != None:
            p.addlc(val[i])
            nodes.append(p.lc)
        i += 1
        if val[i] != None:
            p.addrc(val[i])
            nodes.append(p.rc)
        i += 1
        n += 1
    return root
```

Building a binary tree



Homework

1. The Python list `val` is defined thus:
`val=[5,8,None,3,6,None,None,2,7]`.
Run the `buildtree()` function to build a tree from this input.
2. Draw the tree.
3. Determine the time and space complexities of the `buildtree()` function.



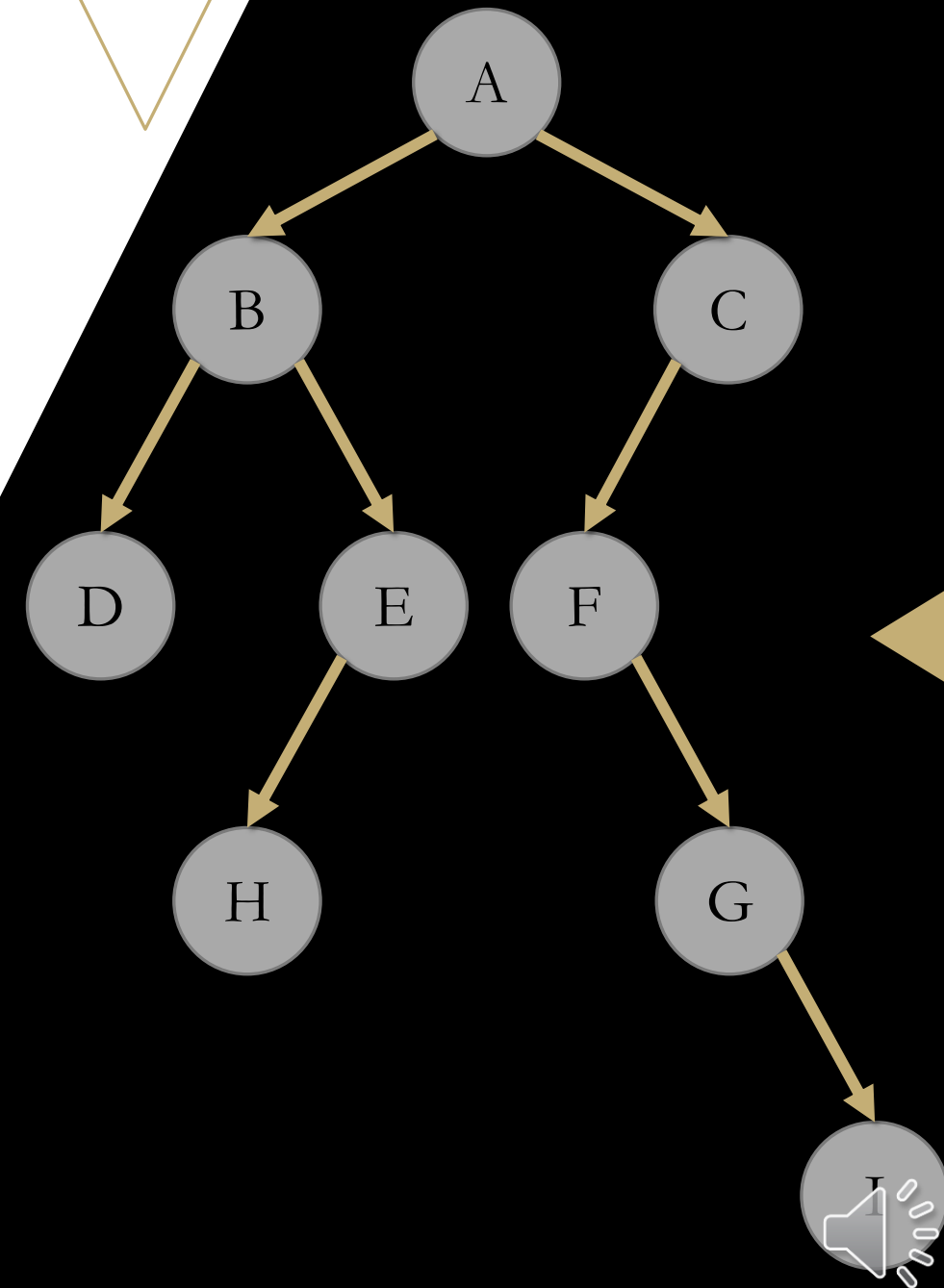
- The function returns a list of two elements.

```
def search(node, target):  
    if node is None:  
        return [None, 0] #no node exists  
    if node.data == target:  
        return [node, 1] #node contains target  
    if node.lc is not None and node.lc.data == target:  
        return [node, 2] #node.lc contains target  
    if node.rc is not None and node.rc.data == target:  
        return [node, 3] #node.rc contains target  
    x = search(node.lc, target)  
    if x[0] is not None: #target is in left subtree  
        return x  
    return search(node.rc, target)
```

Searching in a binary tree



1. Run the search() function on the tree shown here, for targets B, F and J. Explain what the returned lists contain for each of the targets.
2. What is the worst case of the search() function?
3. Determine the time and space complexity of the worst case.





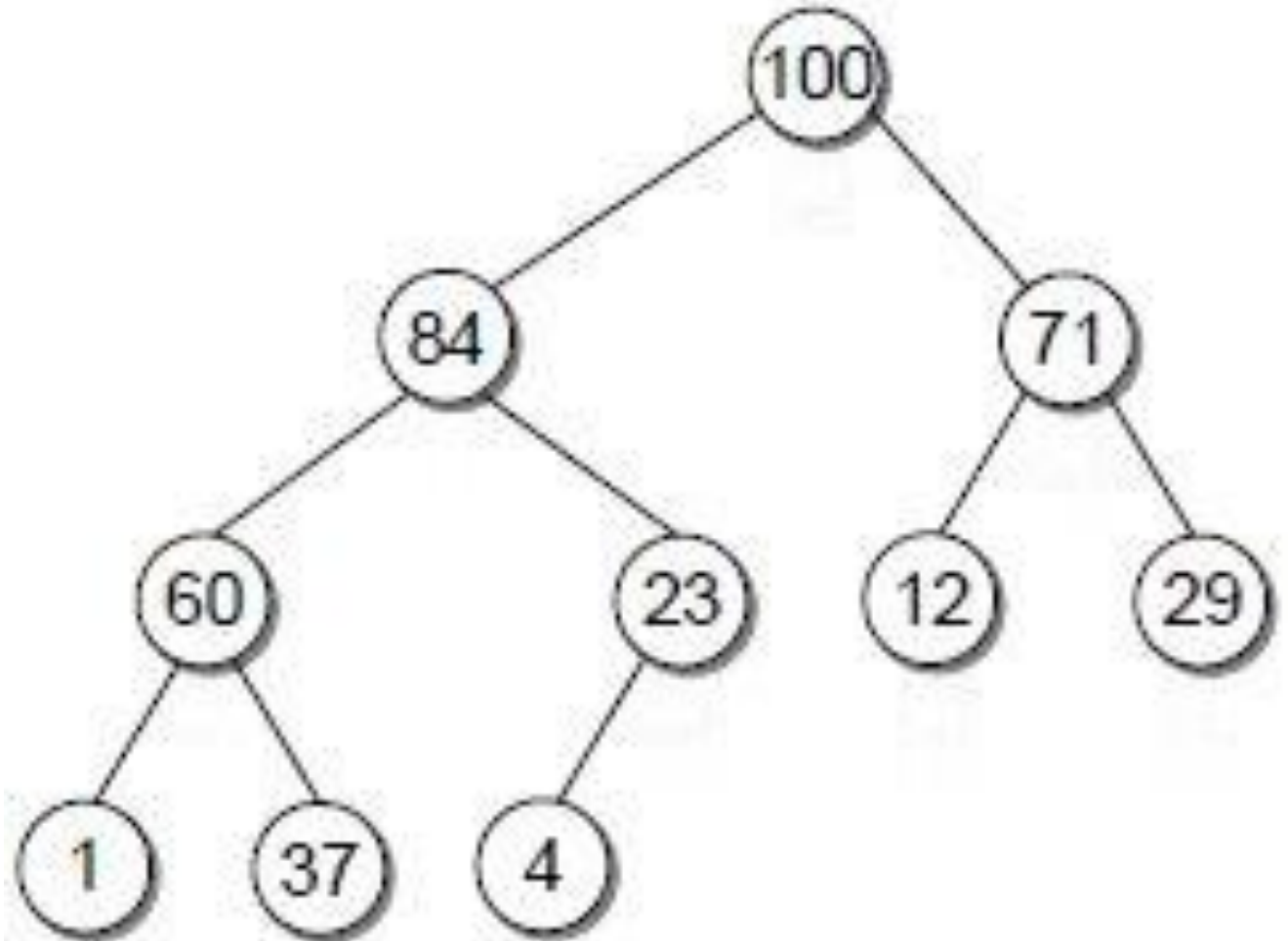
Heaps

- A heap is an **almost complete binary tree** in which the nodes are organized based on their values.
- There are two variants of the heap structure.
 - A **max-heap** has the property, known as the **heap order property**, that dictates that each non-leaf node contains a value greater than the values in its children. The largest value is stored in the root.
 - For the **min-heap**, the heap order property dictates that each non-leaf node contains a value smaller than the values in its children. The smallest value is stored in the root.



Max-heap

In the remainder of this discussion, we will be concerned with max-heaps only.





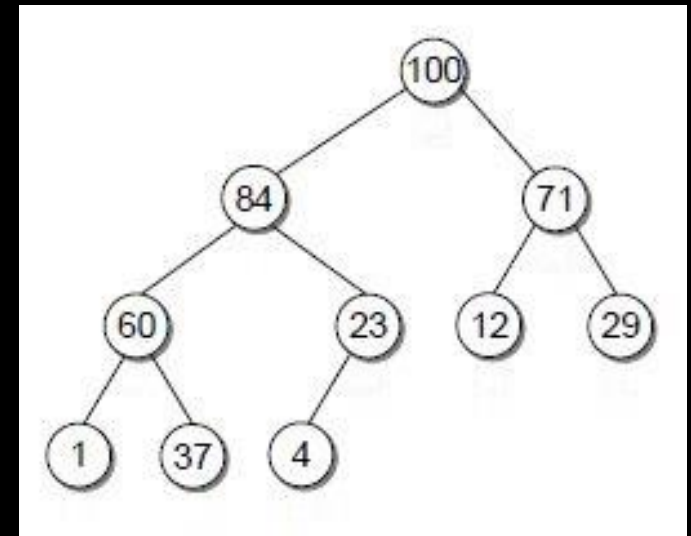
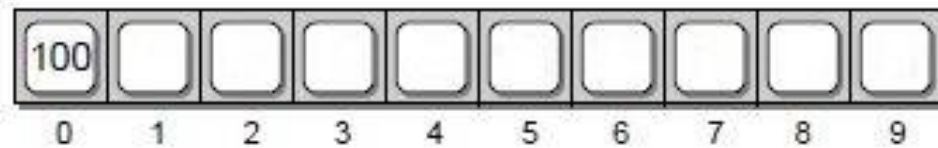
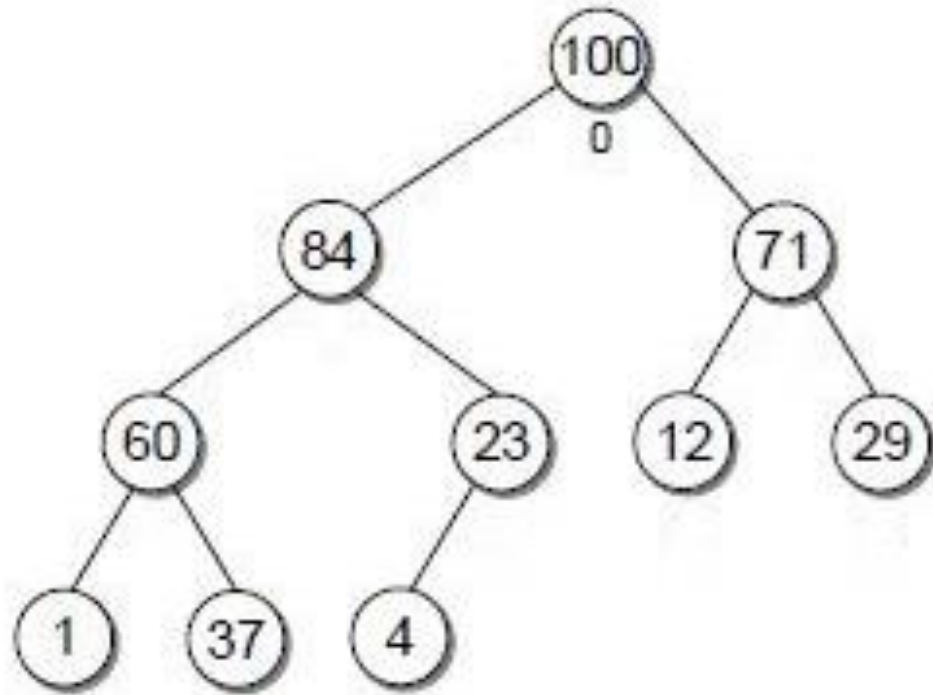
Storage of ACBT in memory

- The heap is an ACBT.
- There are no gaps in between nodes of an ACBT.
- The Python list or an array can be used to store a heap.
- We place the root at index 0 of the list.
- If the parent is at index i , the left child is placed at index $2i + 1$, and the right child at $2i + 2$.
- If a node is at index j , its parent can be retrieved from $\lfloor (j - 1) / 2 \rfloor$.



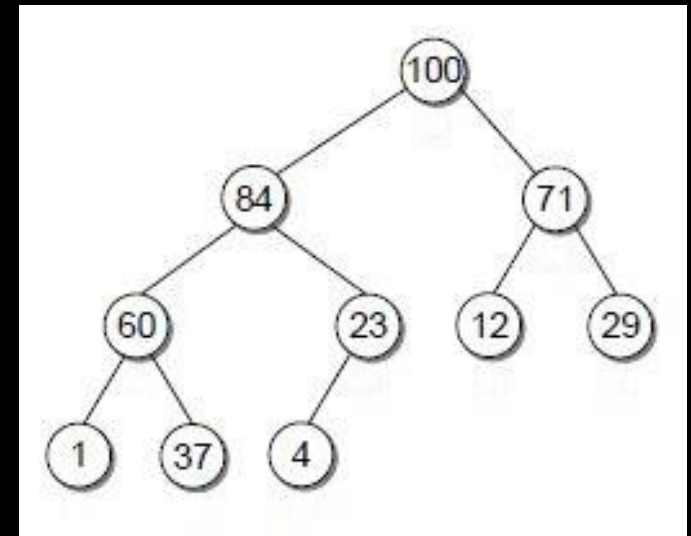
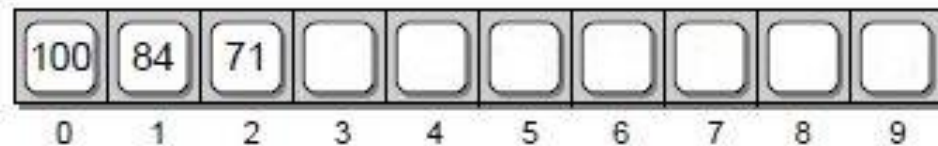
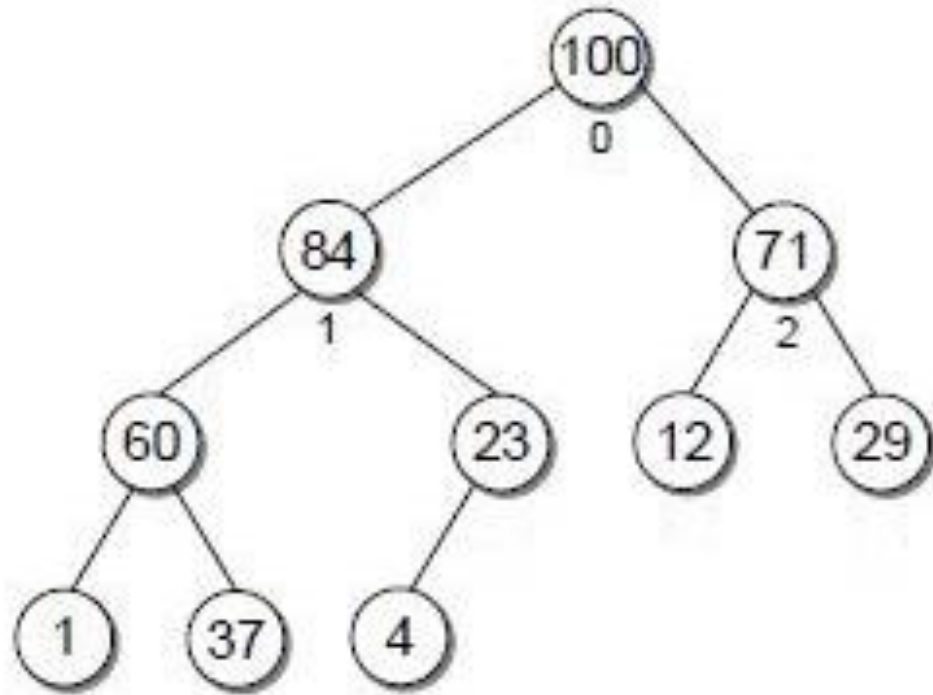
Storing a heap

Store the heap in a list.



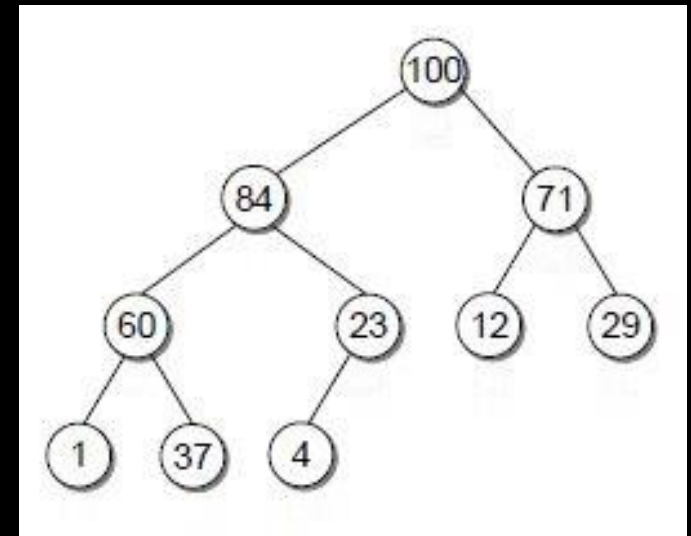
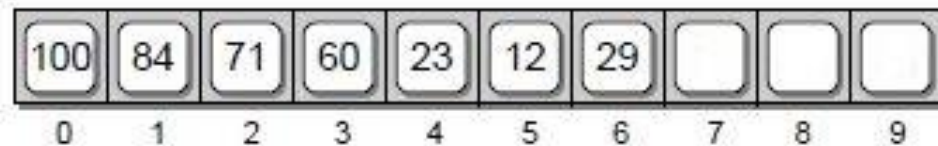
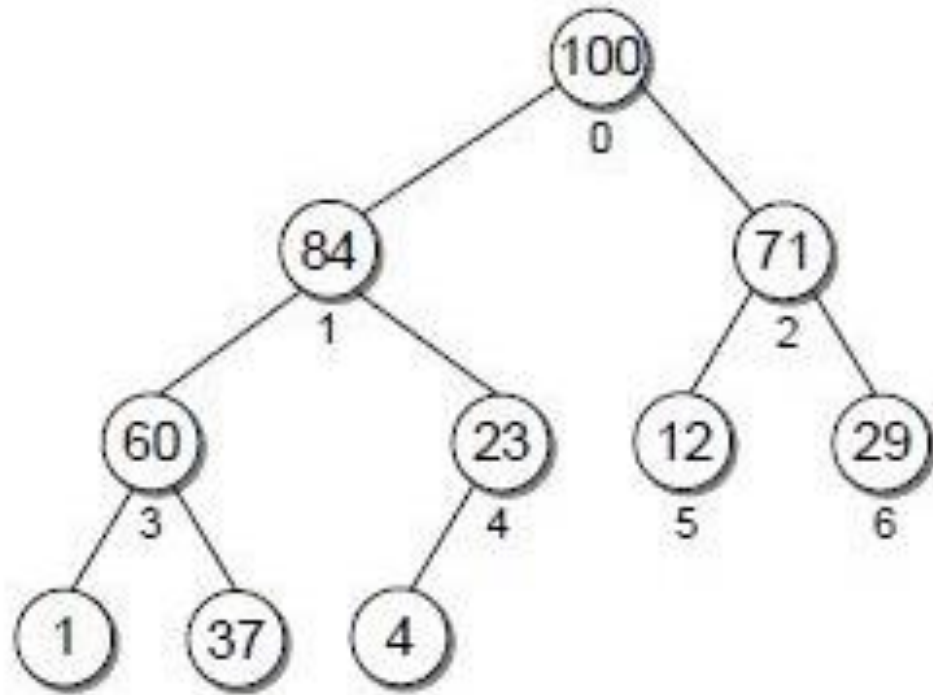
Storing a heap

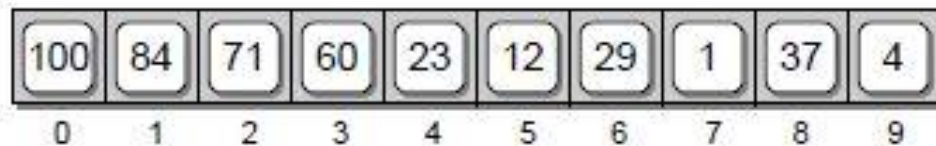
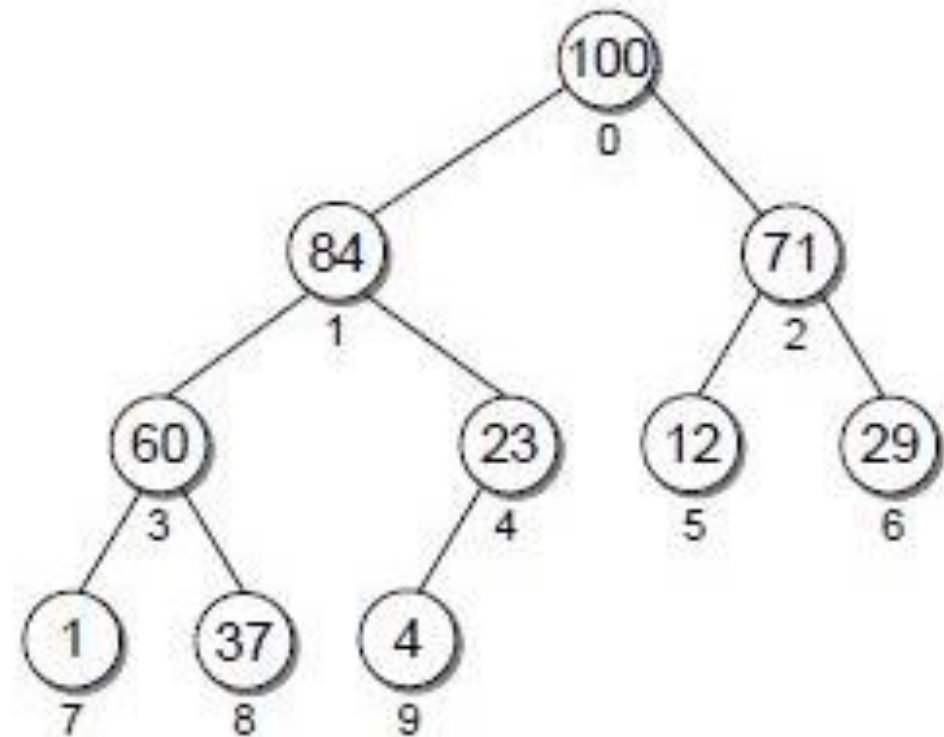
Store the heap in a list.



Storing a heap

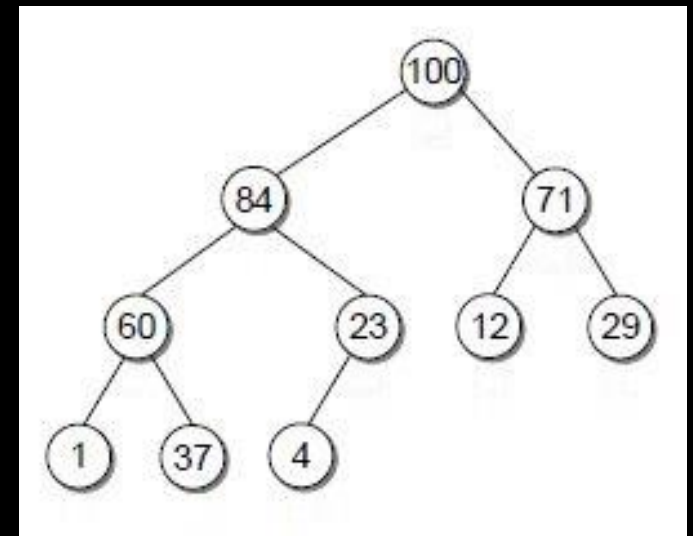
Store the heap in a list.



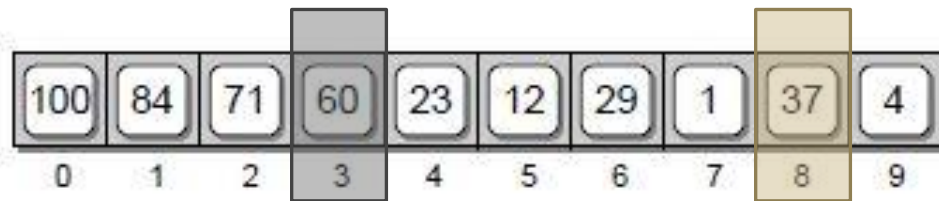


Storing a heap

Store the heap in a list.



Find the parent of 37.

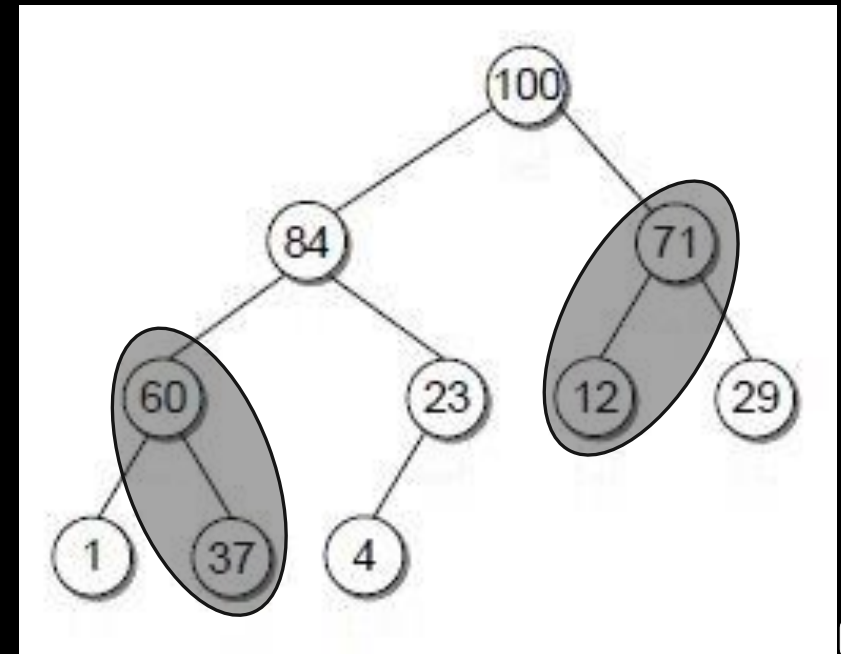


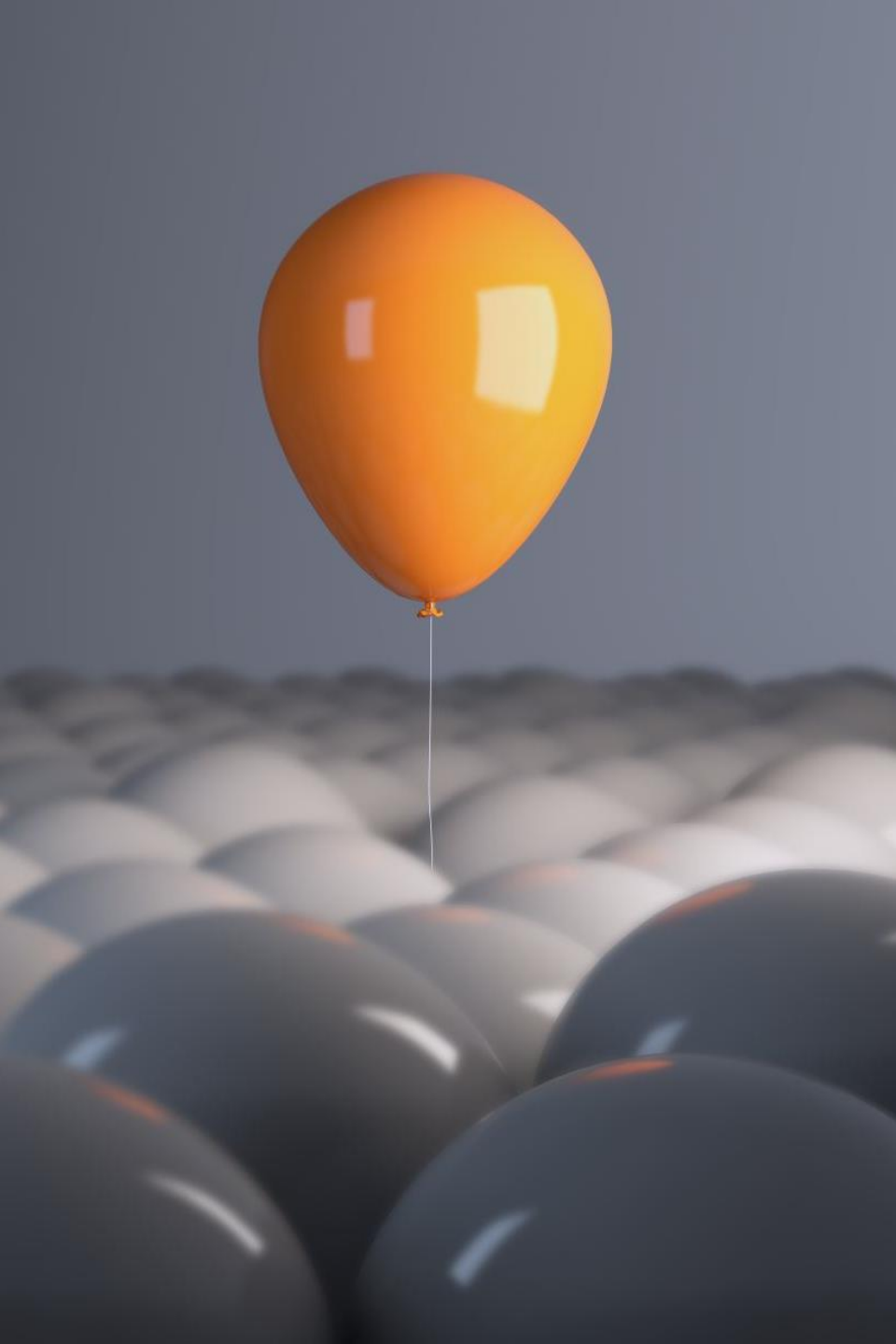
Find the parent of 12.



Finding the parent

Parent of j is at $\lfloor (j-1)/2 \rfloor$.

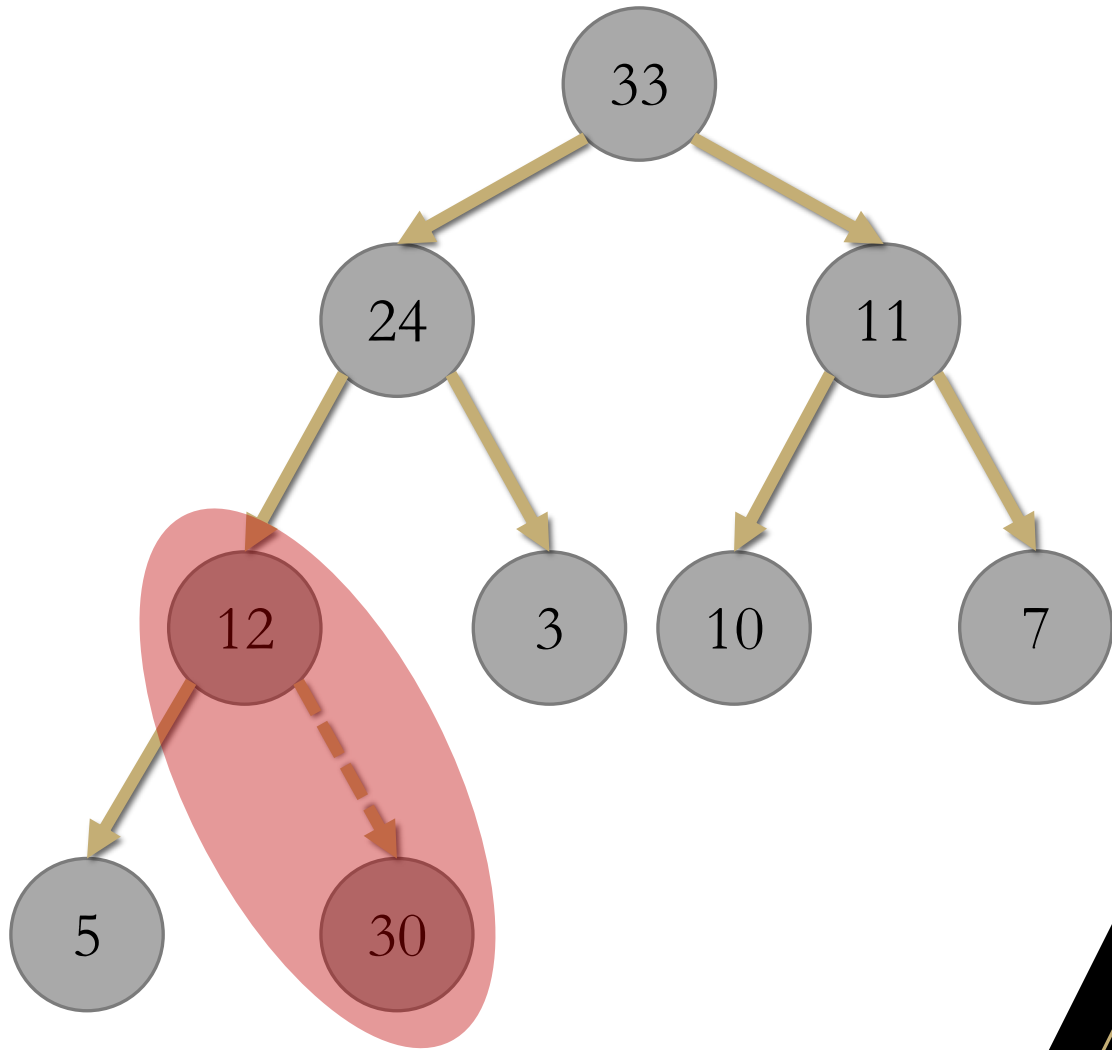




Adding to a heap

- Append the new element to the end of the list, so that the ACBT is preserved.
- Make the new element **rise** to restore heap order property.
- **Rising:**
 - Repeat as long as node's value is greater than parent's value or you do not reach the root:
 - Swap values of the node and its parent.

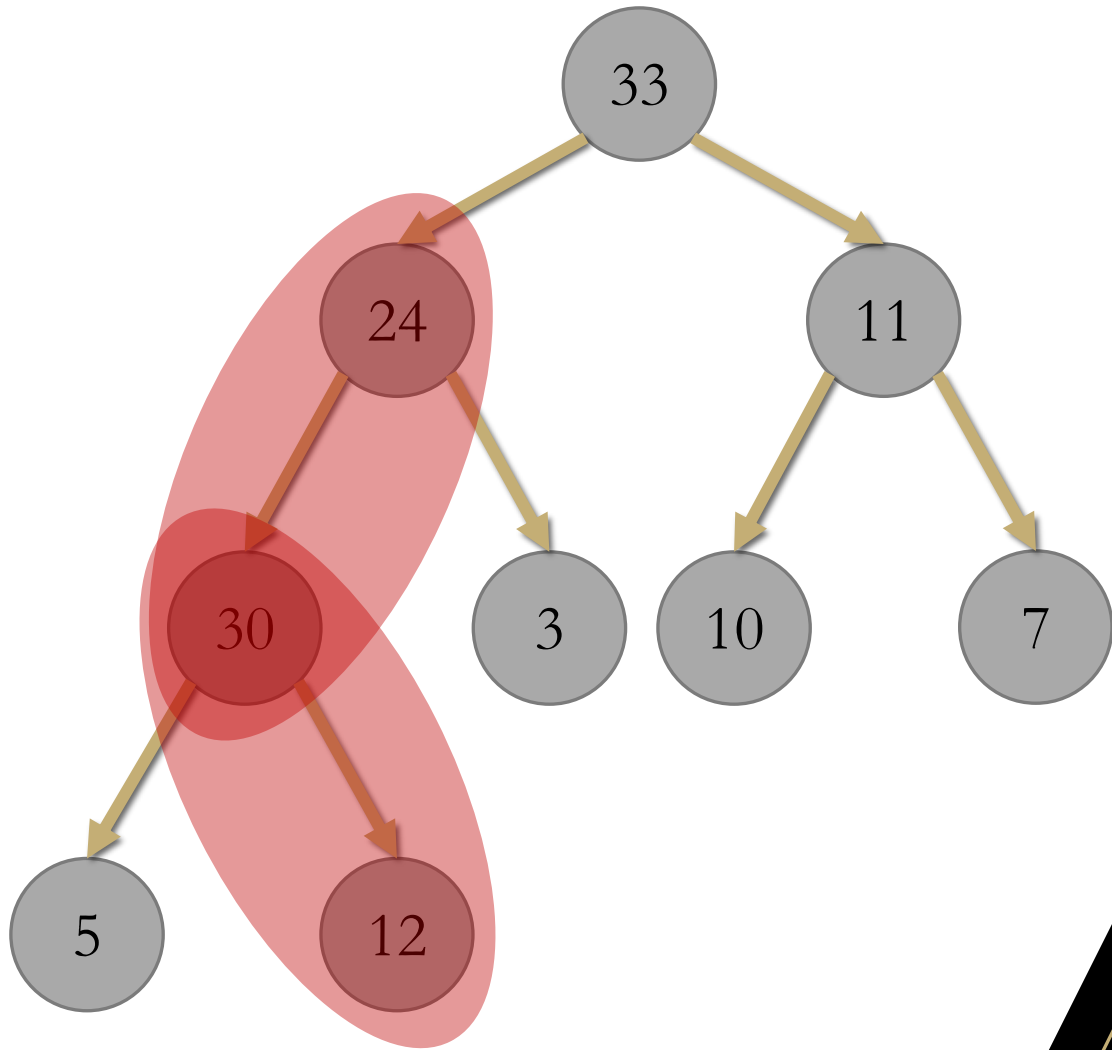




Adding to a heap

Add 30.

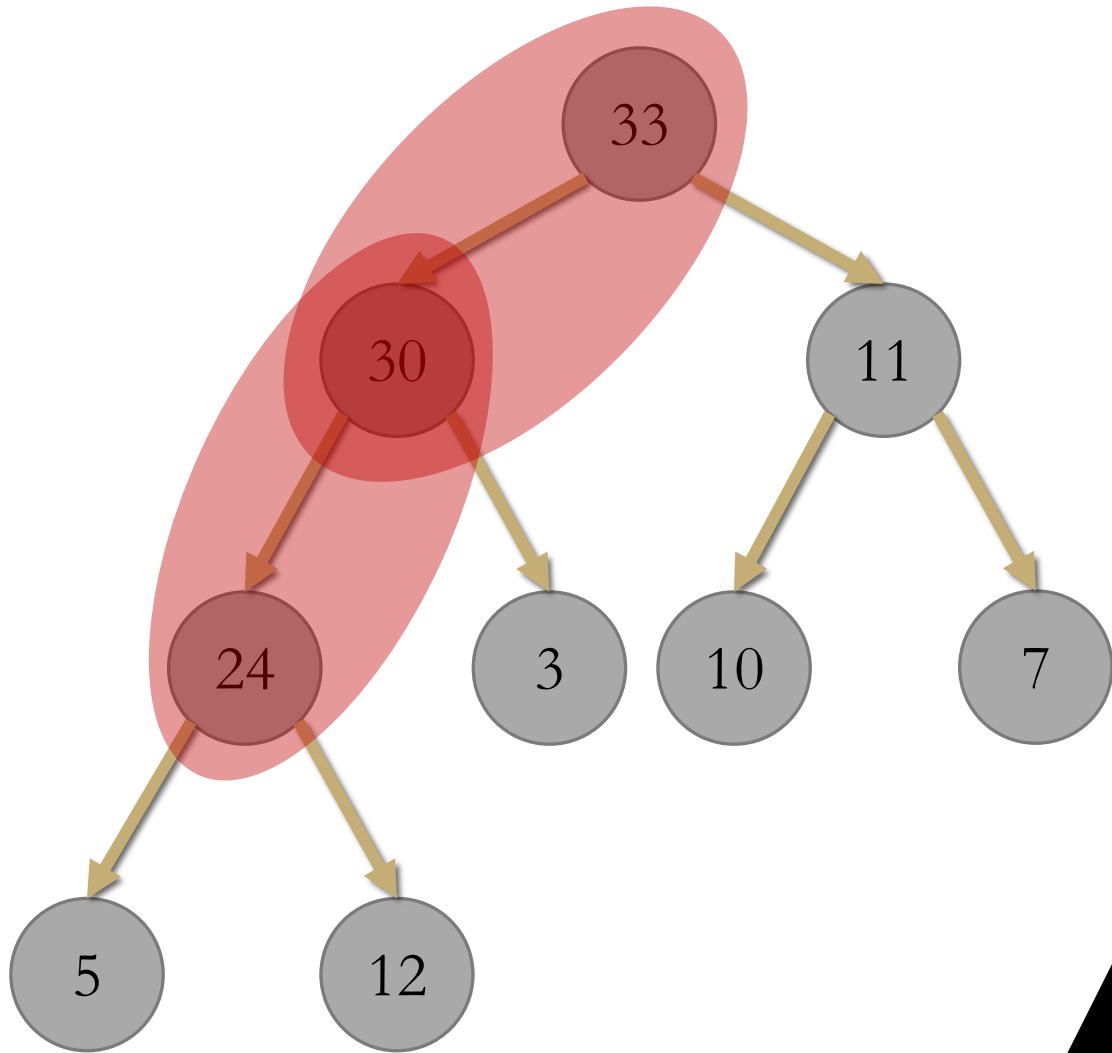




Adding to a heap

Add 30.

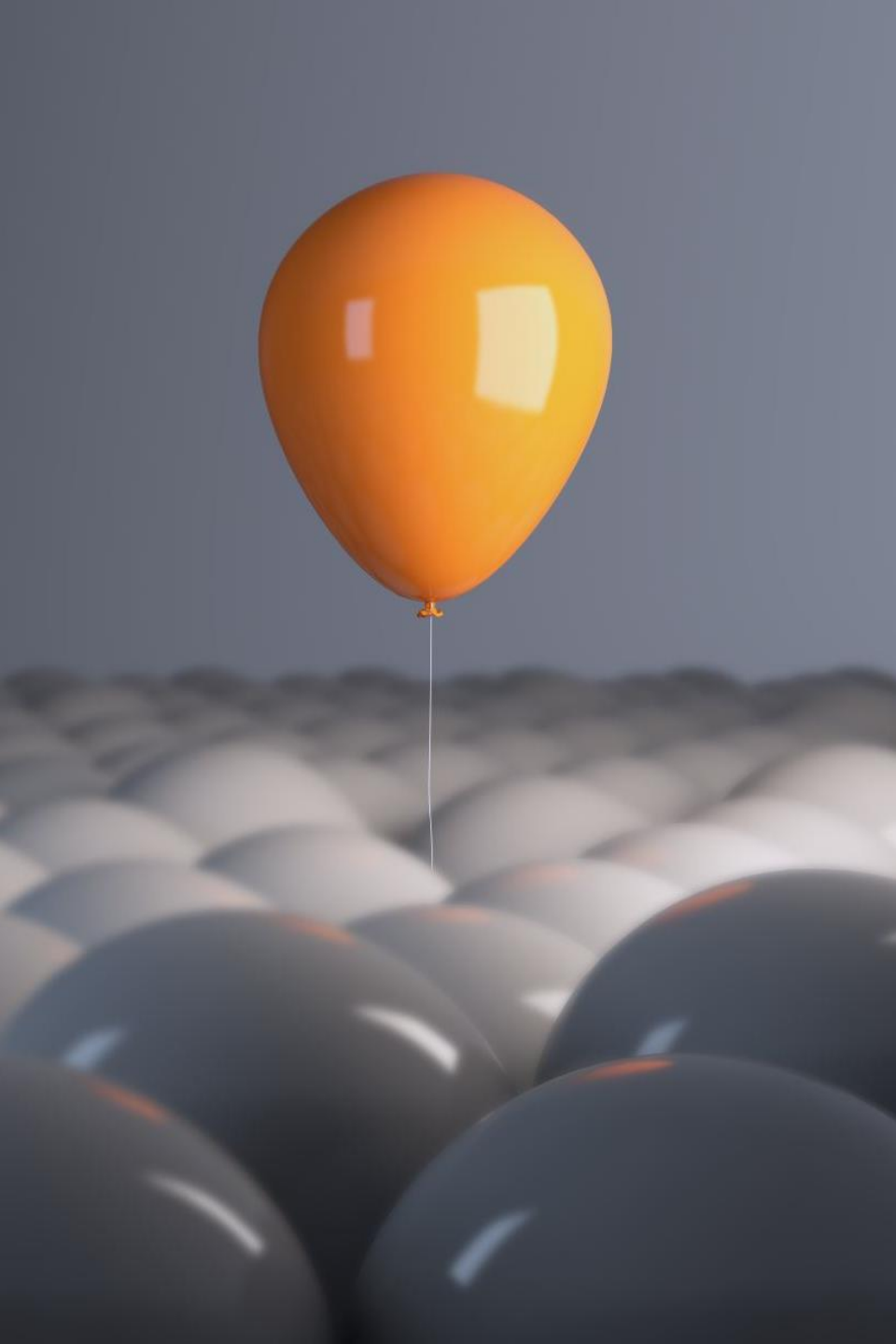




Adding to a heap

Add 30.





Analysis of the rise function

- The worst case happens when a node must rise all the way to the root.
- Height of a PBT of n nodes is $\log_2(n + 1)$.
- Height of an ACBT of n nodes is $O(\log n)$.
- In the worst case, the maximum number of swaps is $\log n$.
- The rise function uses a temporary variable to help in swapping.

Time: $O(\log n)$

Space: $O(1)$

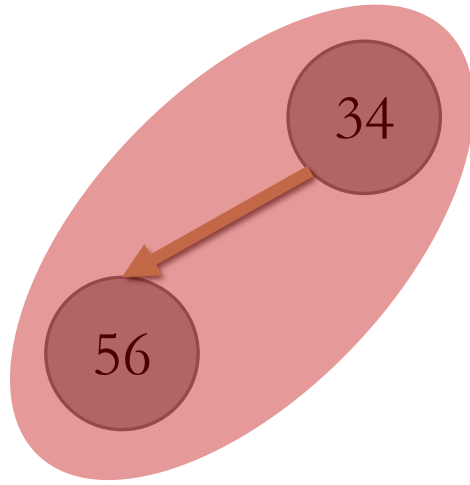


A close-up, high-resolution photograph of dark brown coffee beans. The beans are piled together, with some in sharp focus in the foreground and others blurred in the background, creating a sense of depth. The lighting highlights the texture and creases of the beans.

Building a heap

- Repeat as long as there are elements:
 - Add this element to the heap as described earlier.



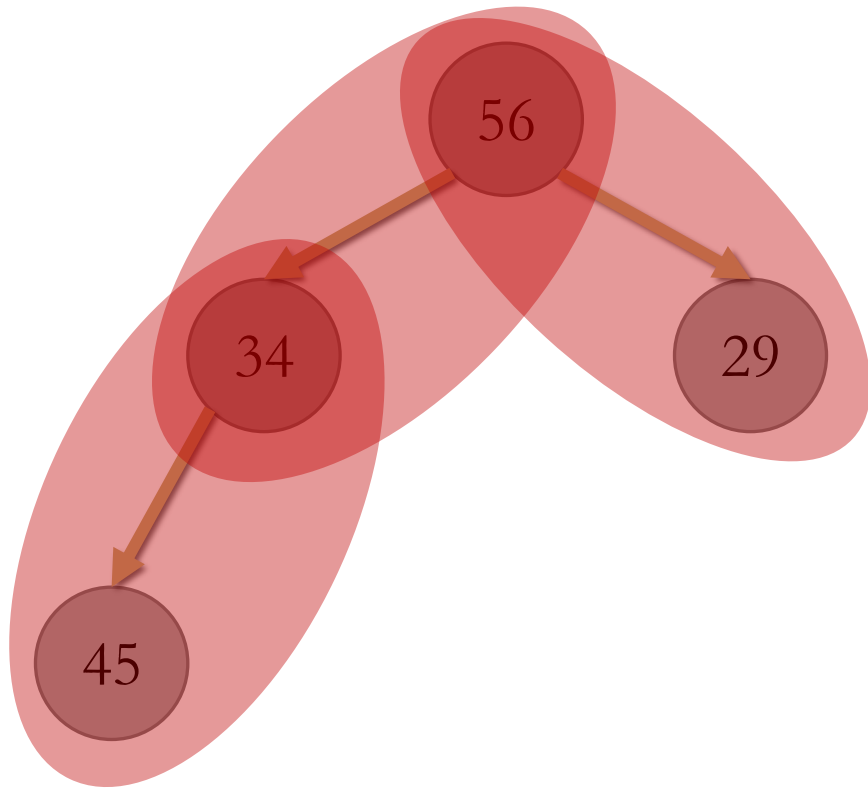


34	56
0	1

Building a heap

Build a heap from
[34,56,29,45,37,68].



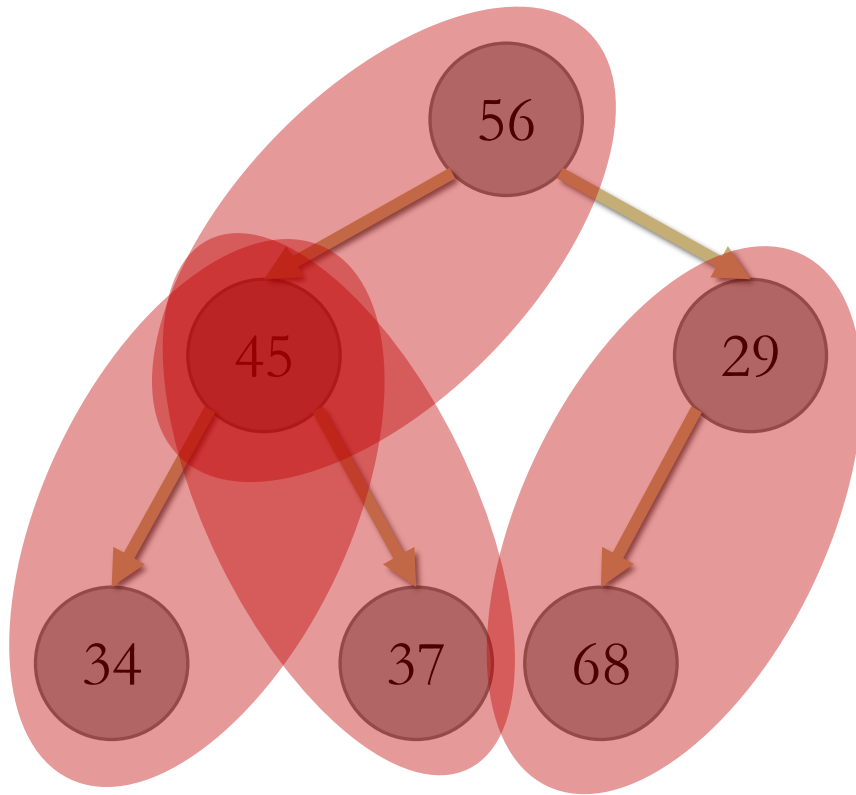


56	34	29	45
0	1	2	3

Building a heap

Build a heap from
[34,56,29,45,37,68].



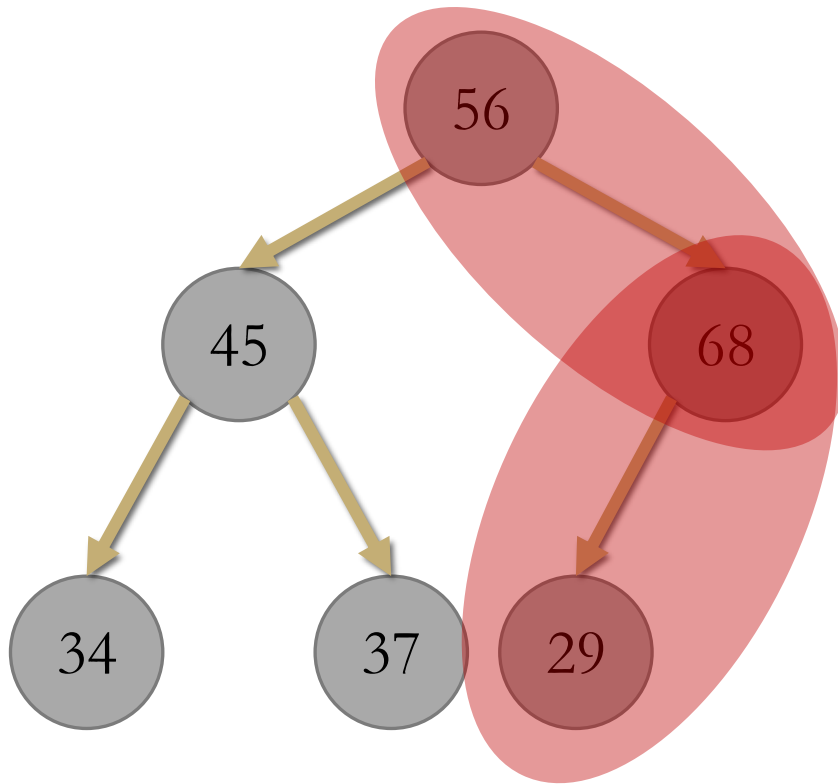


56	45	29	34	37	68
0	1	2	3	4	5

Building a heap

Build a heap from
[34,56,29,45,37,68].



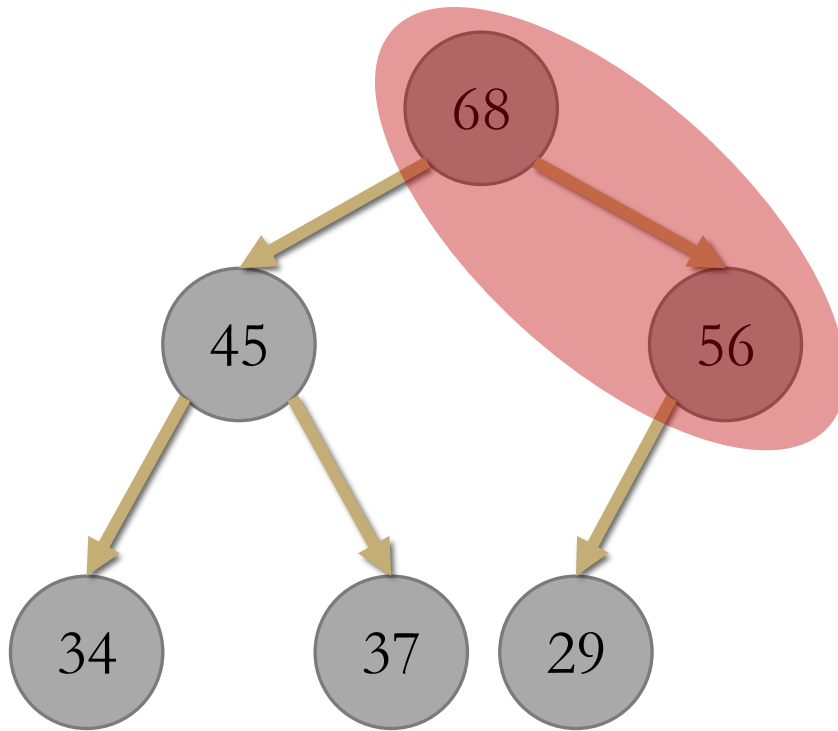


56	45	68	34	37	29
0	1	2	3	4	5

Building a heap

Build a heap from
[34,56,29,45,37,68].





68	45	56	34	37	29
0	1	2	3	4	5

Building a heap

Build a heap from
[34,56,29,45,37,68].



Homework

1. The Python list `val` is defined thus:
`val=[5,8,1,3,6,9,2,7,4]`. Build a heap from this input.
2. Show how this heap is stored in memory.
3. What is the time and space complexity of building a heap?



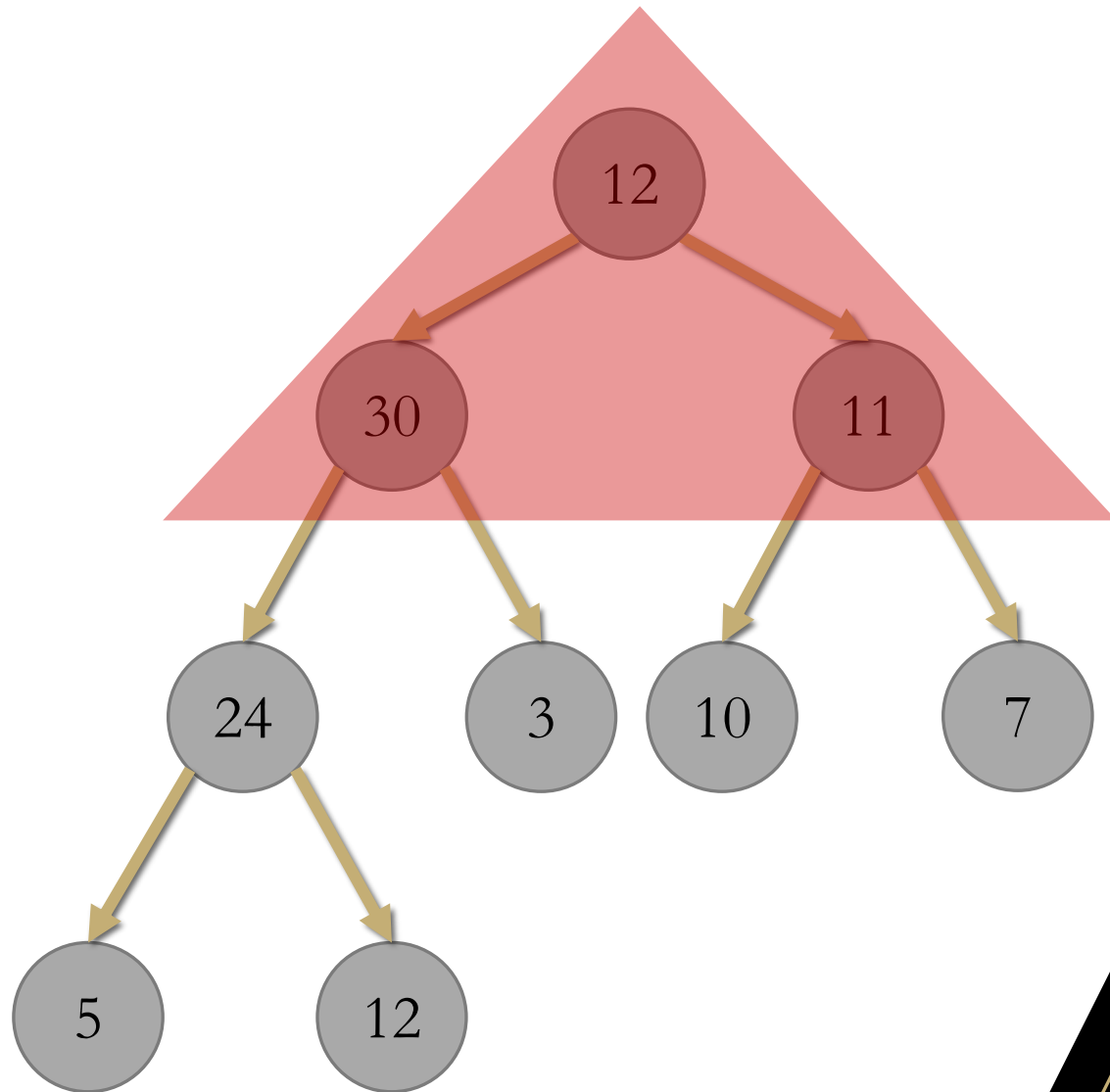


Deleting from a heap

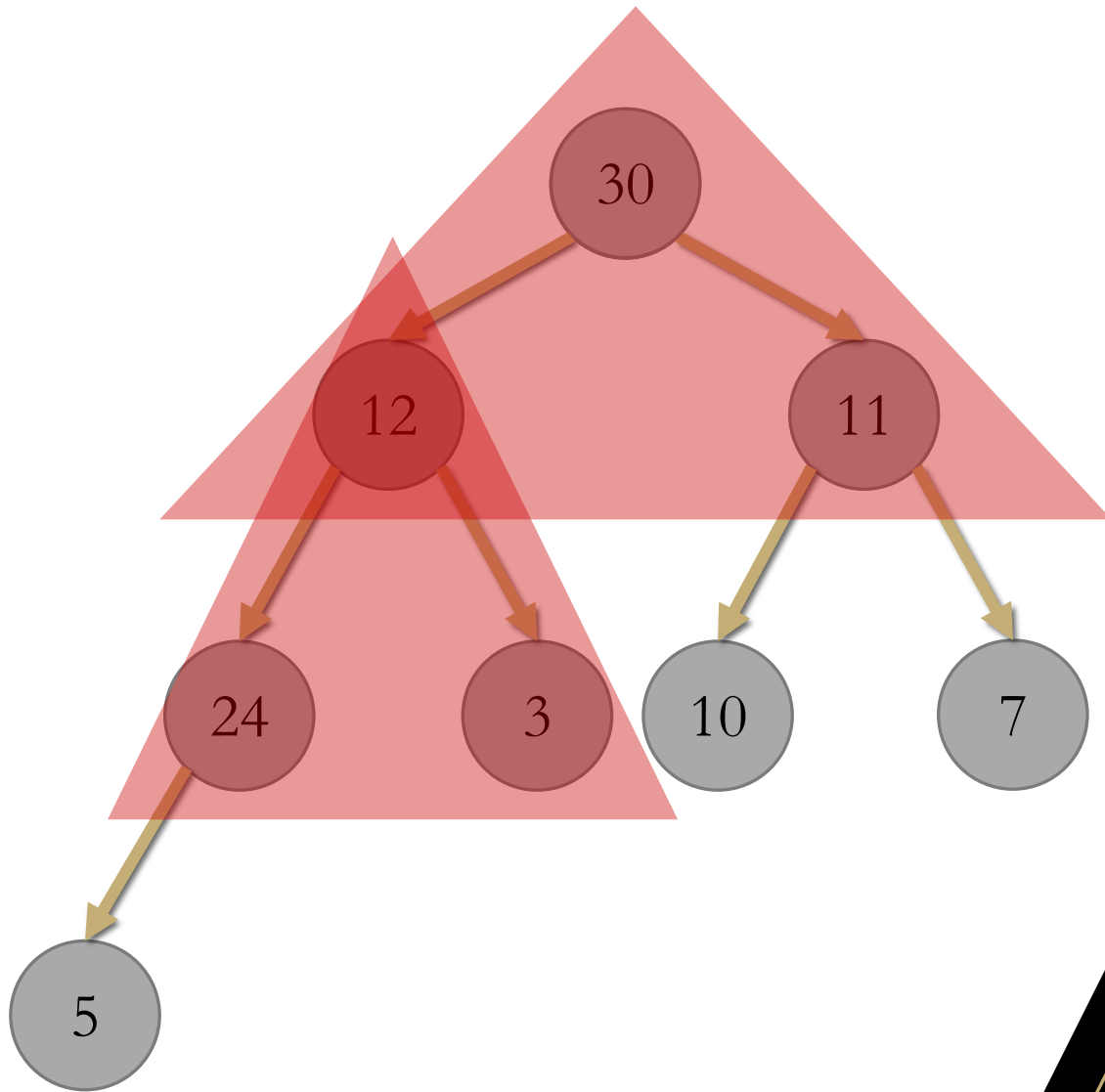
- Always delete the root.
- Make the last element the new root, so that the ACBT is preserved.
- Make the new root **sink** to restore heap order property.
- **Sinking:**
 - Repeat as long as the node's value is less than value of any of its children or you do not reach a leaf:
 - Determine the child with the greater value, x.
 - Swap values of the node and x.

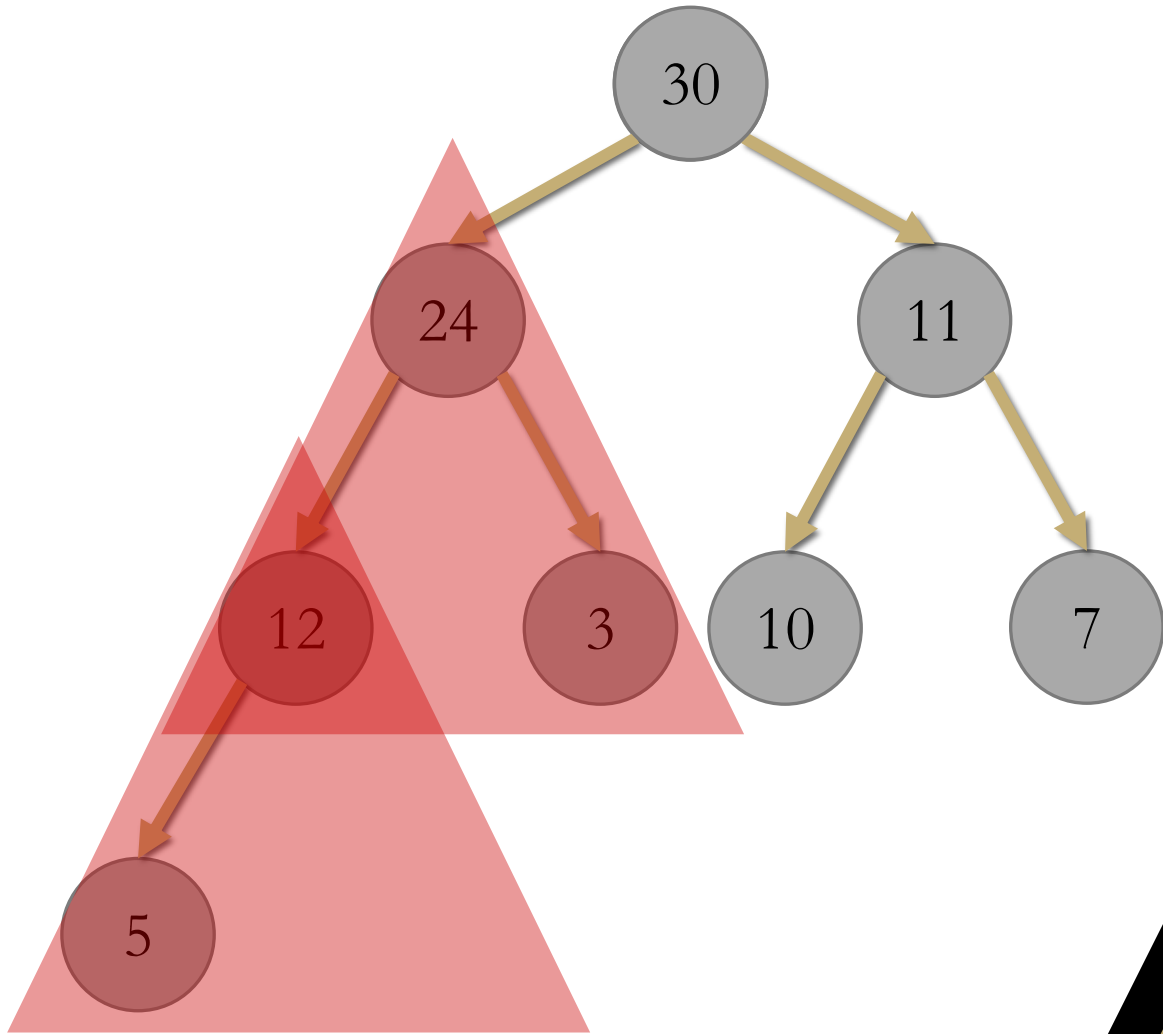


Deleting from a heap



Deleting from a heap





Deleting from a
heap





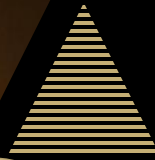
Analysis of the sink function

- The worst case happens when a node must sink all the way to a leaf.
- Height of a PBT of n nodes is $\log_2(n + 1)$.
- Height of an ACBT of n nodes is $O(\log n)$.
- In the worst case, the maximum number of swaps is $\log n$.
- The sink function uses a temporary variable to help in swapping.

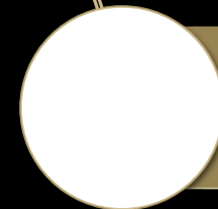
Time: $O(\log n)$

Space: $O(1)$





We were introduced to breadth-first tree traversal algorithm.



We were introduced to the heap.



We learned how to add to, build and delete from a heap.



We examined all codes for time and space complexities.

So, what did we learn today?



Things to do

Read the book!

Email suggestions on content or quality of this lecture at uroojain@neduet.edu.pk

Note your questions and put them up in the relevant online session.

